

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – CONCEPT MAPPING

Course Code	CSA0314	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 1: Concept Mapping
Weightage	35% of CIA	Total Marks	100
CO4 Statement	Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)		
Student Name	ARUTHRAN T	Reg. No.	192525416
Section/Batch	Slot C /	Date	03-10-2026

Code of Conduct:

I, ARUTHRAN T, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand	

Efficient Data Structure Selection for Student Marks Management

PROBLEM STATEMENT

A student management system must store and manage the marks of 1000 students efficiently. Apply primitive and non-primitive data structure concepts to select a suitable structure, and justify the choice by its efficiency.

1000

students, one mark each: efficient storage and management required

STUDENT NAME

Aruthran T

REGISTER NUMBER

192525416

DEPARTMENT

AI/ML

SUBJECT

DATA STRUCTURE

WORKFLOW

1000 Students



Marks Storage



Search



Update



Calculate Results

Data Structure Concepts

Data structure

A systematic way of organising and storing data so it can be accessed and modified efficiently.

Primitive

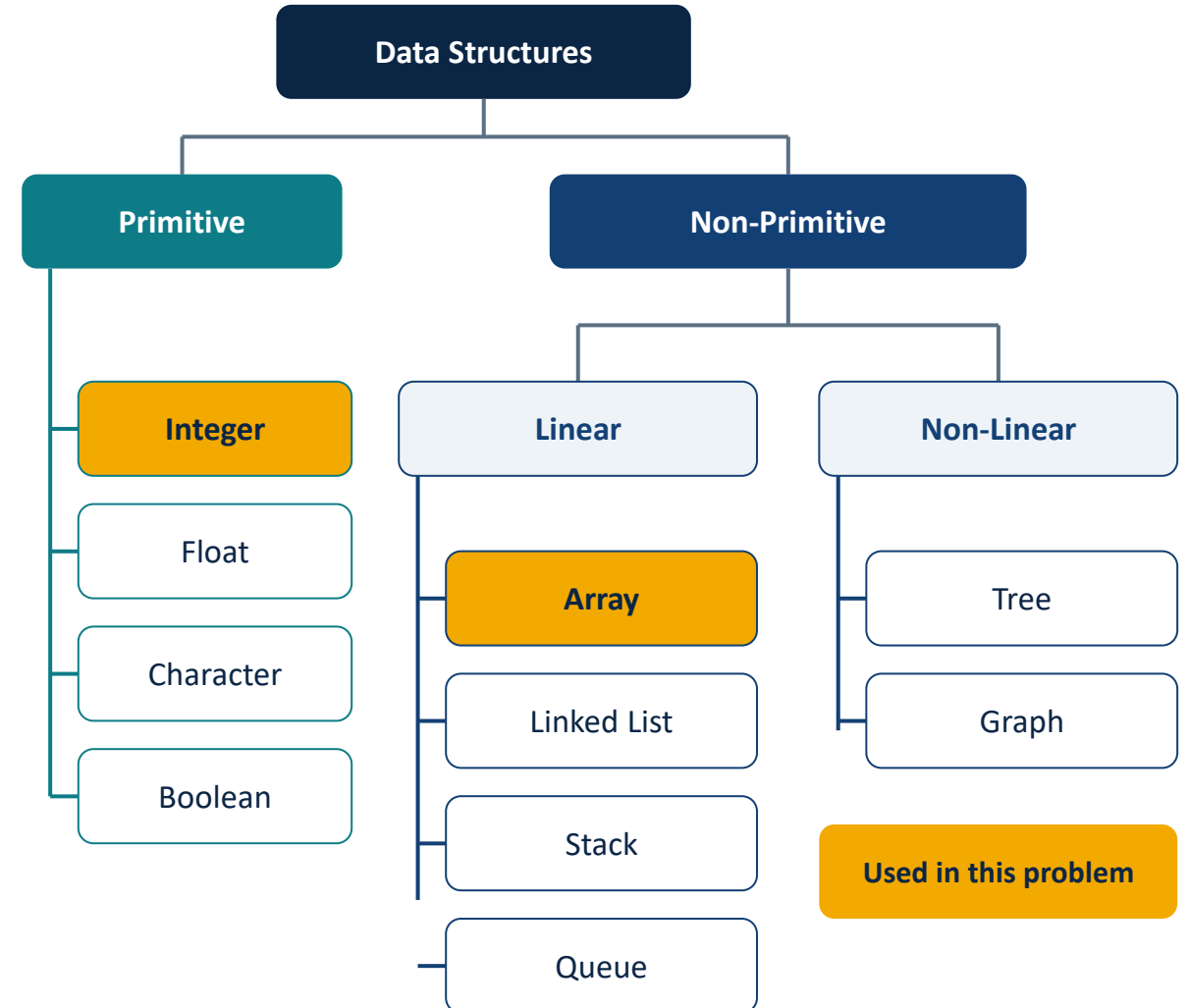
Basic built-in types that hold a single value: int, float, char, boolean.

Non-primitive

Built from primitives to organise many values: Array, Linked List, Stack, Queue, Tree, Graph.

APPLIED TO THIS PROBLEM

Each mark is a whole number, so it is an **int** (primitive). An **array** (non-primitive) organises the 1000 values: **int marks[1000]**



Requirement Analysis & Options

What the system must do

- 1 Store 1000 marks
- 2 Access an individual student's mark
- 3 Update marks
- 4 Search marks
- 5 Calculate average
- 6 Find maximum / minimum
- 7 Traverse all marks
- 8 Use memory efficiently
- 9 Keep implementation simple

Teal = operations, blue = resource goals

Candidate structures

Structure	Main property	Suitability	Reason
Array	Indexed access	High	Direct access by position
Linked List	Dynamic nodes	Moderate	Element i needs traversal
Stack	LIFO	Low	Only top is accessible
Queue	FIFO	Low	Only front/rear accessible
Tree	Hierarchical / search structure	Not necessary	No hierarchy or ordering needed

Selection depends on the required operations. This workload is dominated by indexed access, update and full traversal of a fixed set of values.

Array Analysis: The Primary Candidate

```
int marks[1000];
```

1 Same-type values

Stores 1000 values of one type

2 Contiguous memory

Elements sit side by side

3 Index-based access

`marks[i]` for $i = 0 \dots 999$

4 Direct / random access

No traversal needed to reach i

5 Simple implementation

One declaration, one loop

6 Low, predictable overhead

Fixed size, no per-element pointer

Array memory layout

Index

0	1	2	3	...	999
---	---	---	---	-----	-----

Marks (int values)

85	72	91	68	...	88
----	----	----	----	-----	----

Byte offset from base (assuming 4-byte int)

+0	+4	+8	+12	...	+3996
----	----	----	-----	-----	-------

```
marks[500]
```

accesses the 501st element directly.

Address = $\text{base} + 500 \times 4 = \text{base} + 2000$ bytes: one calculation, no traversal.

Efficiency & Complexity

Operation	Array	Linked List
Access by index	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$
Update by index	$O(1)$	$O(n)$
Traversal	$O(n)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$

Operations needed for this problem

Access
 $O(1)$

Update by index
 $O(1)$

Search
 $O(n)$

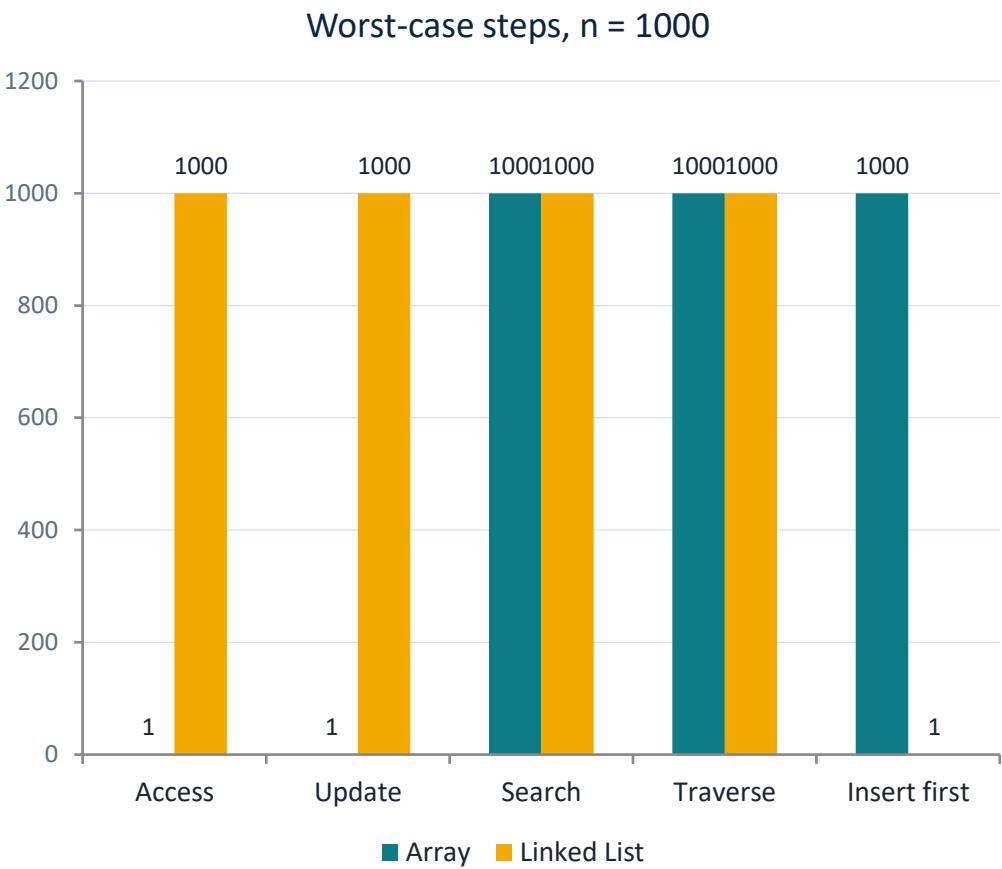
Average
 $O(n)$

Maximum
 $O(n)$

Minimum
 $O(n)$

Traversal
 $O(n)$

Array indexing makes student-specific mark retrieval a single $O(1)$ step.



Illustrative worst-case element visits or shifts: shows order of growth, not measured time. Search assumes unsorted marks; insertion at the front of an array shifts n elements.

Memory & Practical Implementation

```
int marks[1000] → 1000 × 4 = 4000 bytes
```

≈ 3.91 KB

This calculation assumes a 4-byte integer; actual memory usage depends on the language and implementation.

Array

- Minimal structural overhead
- Fixed-size memory requirement

Linked list

- Pointer stored per node
- Dynamic allocation
- ≈16 B/node (4 B int + 8 B pointer + padding) ≈ 15.63 KB

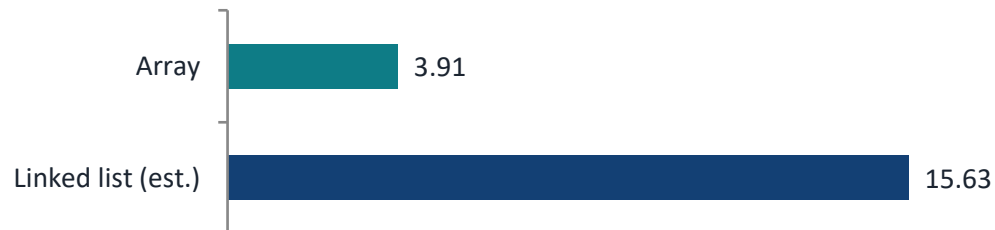
```
#include <stdio.h>
```

```
int main() {  
    int marks[1000], sum = 0;  
  
    for(int i = 0; i < 1000; i++) {  
        scanf("%d", &marks[i]);  
        sum += marks[i];  
    }  
  
    printf("Average = %.2f", sum / 1000.0);  
    return 0;  
}
```

How it works

- The loop stores each mark at marks[i] and adds it to sum: O(n).
- sum / 1000.0 forces floating-point division, so the average keeps its decimals.
- Memory is one 4000-byte block; sum stays within int range for marks up to 100.

Estimated memory for 1000 marks (KB)



Why Array is Selected

SELECTED: Integer Array – marks [1000]

- 1 O(1) indexed access to any student's mark
- 2 O(1) update by index
- 3 Efficient sequential processing (sum, max, min)
- 4 Low overhead and predictable memory (≈ 3.91 KB)
- 5 Simple implementation: one declaration, one loop
- 6 Appropriate for a fixed collection of 1000 marks

Limitation: if the number of students changes frequently or frequent insertion/deletion is required, a dynamic data structure may be more appropriate.

Criterion	Array	Linked List	Stack	Queue
Direct access by index	2	1	0	0
Update by index	2	1	0	0
Traversal, avg, max, min	2	2	1	1
Memory overhead	2	1	2	2
Simplicity for this task	2	1	0	0
Total (max 10)	10	6	3	3

Scoring assumptions: 2 = meets the need directly, 1 = possible with extra cost, 0 = not directly supported. Stack and Queue are scored as access-restricted structures, assumed array-backed. Scores are qualitative and specific to this workload, not a general ranking.

Conclusion & References

- 1 Primitive data types represent individual values.
- 2 Non-primitive structures organise multiple values.
- 3 Integer is appropriate for representing whole-number marks.
- 4 An integer array efficiently stores marks of 1000 students.
- 5 Array indexing provides $O(1)$ access.
- 6 Traversal, search, average, maximum and minimum require $O(n)$.
- 7 The choice is based on the actual requirements and operation efficiency.

$O(1)$

indexed access and update

$O(n)$

search, average, max, min,
traversal

≈ 3.91 KB

for 1000 marks (4-byte int)

Data structure selection should be based on workload, memory requirements, and required operations.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Weiss, M. A. (1997). Data Structures and Algorithm Analysis in C (2nd ed.). Addison-Wesley.

Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.

Horowitz, E., Sahni, S., & Anderson-Freed, S. (2008). Fundamentals of Data Structures in C (2nd ed.). Silicon Press.